

Integration Testing: Example and Explanation

Purpose: Verify the interaction between multiple components (e.g., ProductService and ProductRepository working with a real database, even if it's an in-memory one). Unlike unit tests, integration tests involve more of the application's context.

Tools: JUnit 5, Spring Boot Test, (optionally) Testcontainers. For this example, we'll use an in-memory H2 database for simplicity and speed.

Process:

1. Set up a Spring Boot test environment that loads a portion or all of the application context.
2. Allow Spring to inject real instances of the components you want to test together (e.g., ProductService and its dependencies).
3. Perform operations that involve the integrated components and assert the outcomes.

1. Project Setup (Dependencies in pom.xml for Maven):

Make sure you have the following dependencies in your pom.xml (or build.gradle):

```
<!-- For Spring Boot Web applications -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<!-- For Spring Data JPA and database interaction -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<!-- H2 Database (in-memory for testing) -->
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>

<!-- Spring Boot Test Starter (includes JUnit 5, Mockito, etc.) -->
```

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

2. Update ProductRepository to extend JpaRepository:

For Spring Data JPA to work, your repository needs to extend JpaRepository.

```
// src/main/java/com/example/demo/repository/ProductRepository.java
package com.example.demo.repository;

import com.example.demo.model.Product;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository; // Add this annotation

// JpaRepository provides CRUD operations and more
@Repository // Mark as a Spring Data JPA repository
public interface ProductRepository extends JpaRepository<Product, String> {
    // JpaRepository already provides findById, findAll, save, deleteById
    // You don't need to declare them explicitly here unless you need custom queries
}
```

3. Update Product Model for JPA:

Add JPA annotations to your Product model to map it to a database table.

```
// src/main/java/com/example/demo/model/Product.java
package com.example.demo.model;

import jakarta.persistence.Entity; // Use jakarta.persistence for Spring Boot 3+
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import java.util.Objects;

@Entity // Marks this class as a JPA entity
@Table(name = "products") // Maps to a table named 'products'
```

```

public class Product {
    @Id // Marks 'id' as the primary key
    private String id;
    private String name;
    private double price;

    // Default constructor required by JPA
    public Product() {}

    public Product(String id, String name, double price) {
        this.id = id;
        this.name = name;
        this.price = price;
    }

    // Getters
    public String getId() { return id; }
    public String getName() { return name; }
    public double getPrice() { return price; }

    // Setters
    public void setId(String id) { this.id = id; }
    public void setName(String name) { this.name = name; }
    public void setPrice(double price) { this.price = price; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Product product = (Product) o;
        return Double.compare(product.price, price) == 0 &&
            Objects.equals(id, product.id) &&
            Objects.equals(name, product.name);
    }

    @Override
    public int hashCode() {
        return Objects.hash(id, name, price);
    }
}

```

```

@Override
public String toString() {
    return "Product{" +
        "id='" + id + '\'' +
        ", name='" + name + '\'' +
        ", price=" + price +
        '}';
}
}

```

4. application-test.properties (Optional but Recommended):

Create src/test/resources/application-test.properties to configure the in-memory database for tests. This ensures tests use a separate, clean database.

```

# src/test/resources/application-test.properties
spring.datasource.url=jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1;DB_CLOSE_ON_EXIT
=FALSE
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=create-drop # Recreates schema for each test run
spring.jpa.show-sql=true

```

5. The Integration Test for ProductService:

```

// src/test/java/com/example/demo/service/ProductServiceIntegrationTest.java
package com.example.demo.service;

import com.example.demo.model.Product;
import com.example.demo.repository.ProductRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;

```

```

import org.springframework.test.context.ActiveProfiles;
import org.springframework.transaction.annotation.Transactional; // Important for
clean state

import java.util.List;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;

// @SpringBootTest loads the full Spring application context
// webEnvironment = SpringBootTest.WebEnvironment.NONE means no web server is
started
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.NONE)
@ActiveProfiles("test") // Activates the application-test.properties configuration
@Transactional // Ensures each test runs in a transaction and rolls back changes
@DisplayName("ProductService Integration Tests")
public class ProductServiceIntegrationTest {

    @Autowired // Spring will inject the real ProductService instance
    private ProductService productService;

    @Autowired // Spring will inject the real ProductRepository instance
    private ProductRepository productRepository;

    @BeforeEach
    void setUp() {
        // Clean up the database before each test to ensure isolation
        productRepository.deleteAll();
    }

    @Test
    @DisplayName("Should create and retrieve a product successfully")
    void createAndGetProduct_shouldWork() {
        // Given
        Product newProduct = new Product("P001", "Integration Laptop", 1500.00);

        // When
        Product createdProduct = productService.createProduct(newProduct);
    }

```

```
// Then
assertNotNull(createdProduct.getId());
assertEquals("Integration Laptop", createdProduct.getName());
assertEquals(1500.00, createdProduct.getPrice());

// Verify it's actually in the database via the service's read method
Optional<Product> foundProduct =
productService.getProductById(createdProduct.getId());
assertTrue(foundProduct.isPresent());
assertEquals(createdProduct, foundProduct.get());
}
```

```
@Test
@DisplayName("Should retrieve all products from the database")
void getAllProducts_shouldReturnAllProducts() {
    // Given
    Product product1 = new Product("P001", "Laptop", 1200.00);
    Product product2 = new Product("P002", "Mouse", 25.00);
    productService.createProduct(product1); // Use service to save
    productService.createProduct(product2);
}
```

```
// When
List<Product> products = productService.getAllProducts();
```

```
// Then
assertNotNull(products);
assertEquals(2, products.size());
assertTrue(products.contains(product1));
assertTrue(products.contains(product2));
}
```

```
@Test
@DisplayName("Should update an existing product in the database")
void updateProduct_shouldUpdateProduct() {
    // Given
    Product originalProduct = new Product("P003", "Old Keyboard", 50.00);
    productService.createProduct(originalProduct);
}
```

```
Product updatedDetails = new Product("P003", "New Keyboard", 75.00);
```

```

    // When
    Optional<Product> result = productService.updateProduct("P003",
updatedDetails);

    // Then
    assertTrue(result.isPresent());
    assertEquals("New Keyboard", result.get().getName());
    assertEquals(75.00, result.get().getPrice());

    // Verify the change is persistent in the database
    Optional<Product> persistedProduct = productRepository.findById("P003");
    assertTrue(persistedProduct.isPresent());
    assertEquals("New Keyboard", persistedProduct.get().getName());
    assertEquals(75.00, persistedProduct.get().getPrice());
}

```

```

@Test
@DisplayName("Should delete a product from the database")
void deleteProduct_shouldDeleteProduct() {
    // Given
    Product productToDelete = new Product("P004", "Monitor", 300.00);
    productService.createProduct(productToDelete);
    assertEquals(1, productRepository.count()); // Verify it's there

    // When
    boolean deleted = productService.deleteProduct("P004");

    // Then
    assertTrue(deleted);
    assertEquals(0, productRepository.count()); // Verify it's gone
    assertFalse(productService.getProductById("P004").isPresent());
}

```

```

@Test
@DisplayName("Should not delete non-existent product")
void deleteProduct_shouldNotDeleteNonExistentProduct() {
    // Given
    productRepository.deleteAll(); // Ensure empty
}

```

```

    assertEquals(0, productRepository.count());

    // When
    boolean deleted = productService.deleteProduct("P999"); // Try to delete
    non-existent

    // Then
    assertFalse(deleted);
    assertEquals(0, productRepository.count()); // Still empty
}
}

```

Explanation of the Integration Test:

1. **@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.NONE):**
 - This is the core annotation for Spring Boot integration tests. It tells JUnit to load the entire Spring application context.
 - `webEnvironment = SpringBootTest.WebEnvironment.NONE` ensures that a web server (like Tomcat) is *not* started, as we are only testing the service and repository layers, not HTTP endpoints.
2. **@ActiveProfiles("test"):**
 - This activates the Spring profile named "test".
 - By having `application-test.properties` in `src/test/resources`, Spring Boot will pick up the H2 database configuration specifically for these tests, ensuring they don't interfere with your main application's database.
3. **@Transactional:**
 - This is crucial for integration tests involving a database.
 - When applied to a test class or method, it ensures that each test method runs within a transaction.
 - Crucially, at the end of each test method, the transaction is **rolled back**. This means any changes made to the database by one test are undone, providing a clean database state for the next test. This ensures test isolation.
4. **@Autowired private ProductService productService;**
 - Unlike unit tests where we use `@InjectMocks` to inject a *mock* service, here we use `@Autowired`.
 - Spring's test context will detect `ProductService` as a Spring `@Service` component and inject a *real* instance of it, along with its *real* `ProductRepository` dependency.

5. **@Autowired private ProductRepository productRepository;**
 - We also autowire the ProductRepository directly. This is often useful in integration tests to perform direct database assertions (e.g., `productRepository.count()`, `productRepository.findById()`) to confirm that the service's actions correctly persisted data.
6. **@BeforeEach void setUp():**
 - Even with `@Transactional`, it's good practice to `deleteAll()` on the repository before each test. This provides an explicit cleanup and ensures that if a test somehow commits a transaction or if `@Transactional` isn't fully effective in a complex scenario, you still start with a clean slate.
7. **Testing Flow:**
 - **Given:** Set up the initial state in the database using the `productService` or `productRepository`.
 - **When:** Execute the method(s) on the `productService` that you want to test.
 - **Then:** Assert the results. This includes checking the return value of the service method and, importantly, verifying the state of the database using the `productRepository` to confirm persistence.

Key Differences from Unit Tests:

- **Scope:** Unit tests focus on a single class; integration tests involve multiple interacting components.
- **Dependencies:** Unit tests mock external dependencies; integration tests use real (or near-real, like in-memory DBs) dependencies.
- **Speed:** Unit tests are very fast; integration tests are slower because they involve more setup and I/O.
- **Context:** Unit tests typically don't load the full Spring context; integration tests do (or a significant portion).

Integration tests are vital for ensuring that your application's components work correctly together and that data flows as expected through different layers.